

技术分享

智能体技术介绍 (下)

AI Agents

下篇 · 协作 / 前沿 / 落地：多智能体、协议、自进化、Computer Use 与工程实践

演进 · 原理 · 前沿 · 落地

中交信科集团星宇公司 · 王宇, 2026-06





本场地图 · 下篇

续上篇：从「单个 agent 怎么工作」，到协作、前沿与落地



1

多智能体协作

五种编排模式 · 何时值得 · 代价几何



2

协议与互联

MCP 连工具 · A2A 连 agent · Skill 打包能力



3

工程驾驭

上下文工程 · Harness · Loop Engineering



4

自进化

让 agent 边跑边变强 + 安全护栏



5

新范式

智能体经济（自己花钱） · Computer Use（像人操作）



6

现实与落地

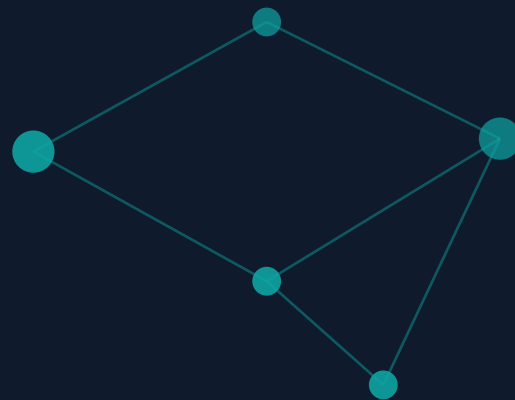
卡点 · 七个坑 · 评估 · 风控 · 决策树

05

FRONTIER 2026

前沿 2026

今年的关键词不是‘更强的模型’，而是协议化、驾驭工程、可靠性与安全。





分工协作

多智能体系统（2024）



三种常见模式

- **主管-执行 (Orchestrator-Worker)**：规划者拆任务，多个执行者并行/串行完成
- **角色分工**：模拟一个团队——程序员 + 测试 + 评审互相补位
- **辩论 / 投票**：多个 agent 各给答案再综合，降低幻觉

+70% 协调良好的多智能体，目标达成率比单 agent 最高可提升约七成（企业基准）。

但是： agent 越多 → 越贵、越难调试、失败点越多。务必‘先把单 agent 做好，再考虑多 agent’。



分工协作 · 进阶 ①

多智能体系统：五种编排模式

1

顺序链 / Pipeline

一个接一个、各司其职。坑：每次交接增加延迟、易丢上下文

2

并行扇出 Fan-out

独立子任务并行跑。前提：分支之间真正互不依赖

3

主管-执行 Supervisor

中心路由器拆任务派活；**框架支持最广**。坑：过度委派（用迭代上限兜住）

4

分层委派 Hierarchical

多个子主管各管一个领域；按复杂度需要才加层级

5

辩论 / 共识 Debate

多 agent 各给答案再综合，降幻觉。代价 $\approx 2.5\times$

6

人在环 HITL

关键外部动作前，插一个人工确认关卡

怎么组合：生产里常叠用——**扇出研究** → **主管质检** → **HITL 把关** → **高风险再共识**。一条铁律：**编排复杂度跟着任务复杂度走**，三个子 agent 能解决的事，别提前上 swarm。



多智能体：什么时候值得，代价是什么

多 agent 不是越多越强——先想清楚为什么要拆，以及拆开后多付的代价。

为什么要拆成多 agent (3 个理由)

- **上下文超限**：连 20 万 token 窗口，也会被研究+编码+测试+部署塞满；拆开让每个上下文聚焦、更准
- **专长分工**：带仓库知识的编码 agent，做代码评审胜过通才
- **并行提速**：真正独立的子任务并行跑，端到端更快

代价与失败面（常被低估）

- **Token 开销**：对等多 agent $\approx +58\%$ ，中心化主管 $\approx +285\%$ （相对调好的单 agent）
- **$O(n^2)$ 失败面**：4 个 agent = 6 个失败点，10 个 = 45 个，超过 8 个测试就盖不全
- **上下文不一致**：两个 agent 定义不同 $\rightarrow$ 冲突，主管也无从裁决
- **难追溯**：并发无单一轨迹，监管域里 swarm 通常不可取

一篇 2026 研究的反直觉结论：给单 agent 同等算力预算，多跳推理任务上常常追平、甚至超过多 agent 系统。铁律——**先把单 agent 做到位**；确实撞到「上下文/专长/并行」的墙，再上多 agent，且从**主管模式**起步。



前沿全景 · 年中复盘

2026 最受关注的六个方向



自进化智能体

从“出厂即定型”到“边跑边进化”



上下文工程

仍是生产可靠性的胜负手



协议层收敛

MCP/A2A 连接 · ACP/UCP 交易标准



安全与治理

投毒·注入·过度授权 + 反 agentwashing



智能体经济

agent 成“买家”，自主支付结算(x402/AP2)



Computer Use

操作无 API 老系统，逼近人类基线



前沿 · 协议

三层栈：让 agent 之间、agent 与工具互联

应用 / 编排层

Orchestration —— 谁来规划、调度整体任务

A2A · 横向

Agent $\leftrightarrow$ Agent：发现彼此、委派、协作 (Google, 2025)

MCP · 纵向

Agent $\leftrightarrow$ 工具/数据：标准化连接 (Anthropic, 2024)

口诀：MCP 纵向(连工具) + A2A 横向(连 agent)，互补不竞争，生产级多智能体两者都要。

治理信号：MCP 已于 2025 年底捐给 Linux 基金会的 Agentic AI Foundation；月下载量已破 9700 万，五大厂 (Anthropic/OpenAI/Google/Microsoft/Amazon) 全支持。另有 IBM/AGNTCY 的 ACP 作为 REST 风格备选。



MCP 详解：AI 世界的‘USB-C’

架构：Host (如 IDE/客户端) ↔ Client ↔ Server (暴露 工具/资源/提示)，全程基于 JSON-RPC 2.0。

tools/list

```
// ① 发现有哪些工具
→ {"jsonrpc":"2.0","id":1,"method":"tools/list"}
← {"jsonrpc":"2.0","id":1,"result":{"tools":[
  {"name":"get_weather","description":"...", "inputSchema":{...}}]}}
```

tools/call

```
// ② 调用工具
→ {"jsonrpc":"2.0","id":2,"method":"tools/call",
  "params":{"name":"get_weather","arguments":{"city":"上海"}}}
← {"jsonrpc":"2.0","id":2,"result":{"content":[{"type":"text","text":"上海 24°C"}]}}
// 出错则返回 error 对象, 如 -32602 Invalid params
```

洞见：一个工具按 MCP 写一次，任何支持 MCP 的模型都能直接用——把 $M \times N$ 的私有对接，变成 $M+N$ 的标准接口。



前沿 · 协议

MCP 进阶：不止工具，还有资源与提示

Tools · 工具

可被模型调用的函数，通常带副作用。例：query_order、create_refund

Resources · 资源

只读上下文数据（文件 / 数据库行 / API 结果）。例：order://12345 订单详情

Prompts · 提示

服务端预置的提示模板 / 工作流。例：一键载入‘标准退货话术’

JSON-RPC 2.0 · 发现并读取资源

```
// initialize 握手，互相声明能力
→ {"method":"initialize"}
← capabilities:{tools,resources}

// resources/list 发现可读资源
→ {"method":"resources/list"}
← [{uri:"order://12345",
    name:"订单12345"}]

// resources/read 取回内容入上下文
→ read {uri:"order://12345"}
← {text:"鞋 ¥299 6/3 已签收"}
```

一次接入，处处可用。 淘宝把‘订单系统’包成一个 MCP server (tools+resources+prompts)，任何支持 MCP 的客服 agent 都能即插即用——把 M×N 私有对接变成 M+N 标准接口。



前沿 · 协议

A2A 详解：让不同厂的 agent 互相协作

AGENT CARD

```
// Agent Card: agent 的“名片”  
// 放在 /.well-known/agent-card.json  
{  
  "name": "BillingAgent",  
  "description": "处理退款与账单",  
  "url": "https://.../a2a",  
  "skills": [{ "id": "refund" }],  
  "authentication": { "schemes": [ "oauth2" ] }  
}
```

任务生命周期 (Task)

submitted → working → **input-required** → completed / failed / canceled

传输：HTTP + SSE + JSON-RPC 2.0；用 Agent Card 做能力发现。

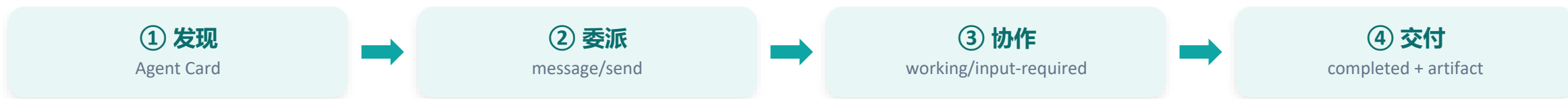
真实案例：销售 agent 经 A2A 把任务路由给 PayPal 的 agent；后者再用 MCP 去调真正的支付工具。

A2A 负责‘agent 之间握手’，MCP 负责‘agent 调工具’。

Agent Card 之于 A2A，就像 OpenAPI 之于 REST——一份机器可读的能力声明，让陌生 agent 也能被发现和调用。



A2A 进阶：两个 agent 的一次完整握手



一次任务委派 (JSON-RPC over HTTP)

```
// 客服 agent → 账单 agent 退款
→ message/send
  {text:"订单12345退款¥299"}
← task: input-required
  {prompt:"请提供退款原因"}
→ message/send
  {text:"尺码不合"}
← task: completed
  {artifact:{refundId:"R-77"}}
```

任务状态机 (Task)

submitted → working → input-required
→ completed / failed / canceled

- 传输: HTTP + SSE + JSON-RPC 2.0
- 用 Agent Card 做能力发现 (置于 /.well-known/)
- 之于 A2A, 如同 OpenAPI 之于 REST——机器可读的能力声明, 让陌生 agent 也能被发现

A2A 管‘agent 之间怎么对话’，MCP 管‘agent 怎么调工具’。上例里账单 agent 收到任务后，自己再用 MCP 去调真正的退款工具——两者叠在一起，才是完整的多智能体系统。



Agent Skill: 把‘怎么做’打包成可复用的本事

一个 Skill = 一个文件夹

- 一份 SKILL.md (说明何时用、怎么做) + 可选脚本 / 模板 / 资源
- **渐进式加载**: 平时只读‘名字+一句描述’, 用到时才载正文——省上下文
- 模型自己按 description 匹配任务, 决定何时用哪个 skill
- **与工具的区别**: 工具 = 一个动作; Skill = 一整套做法 / 流程知识

一个 Skill 的目录结构

```
taobao-refund/      退货退款技能
├─ SKILL.md          触发说明+步骤
├─ refund_policy.md   各类目规则
├─ scripts/
│   └─ check_window.py 算退货期
└─ templates/
    └─ reply.txt        话术模板
```

元数据

总在内存



正文

用时载入



资源

按需读取

工具让 agent‘能做一个动作’, skill 让它‘会做一整件事’。把专家经验写进 SKILL.md, 胜过把所有规则硬塞进 system prompt——需要时才加载, 既省上下文又好维护。



一个真实 Skill 长什么样：淘宝退货

SKILL.md (节选)

```
---  
name: 淘宝退货处理  
description: 用户要退货 /  
退款 / 不想要了 时使用; 判断  
可否退、发起退款、生成话术。  
---  
## 步骤  
1. query_order 查签收日期  
2. 读 refund_policy.md 判规则  
3. 跑 check_window.py 算期内  
4. 可退 → create_refund (确认)  
5. 用 reply.txt 生成回复
```

从触发到执行

- 1 用户说‘鞋子想退’
- 2 模型读到 description → 命中‘淘宝退货’Skill
- 3 加载 SKILL.md 正文 + 按需读 policy / script
- 4 照步骤执行（其中调 MCP 工具 + 人工确认）
- 5 产出‘可退’结论 + 标准话术

写一次，处处复用、口径统一、好维护。所有客服 agent 共用同一套退货逻辑；要改规则只改 SKILL.md，不用动模型、也不用改 prompt。



前沿 · 可靠性

上下文工程 + 确定性护栏



上下文工程

- 动态组装窗口：系统指令+工具+记忆+状态
- 压缩历史（滚动摘要），只留有用的
- 按相关性检索记忆，用完即弃
- 目标：在对的时间，给对的信息



确定性护栏（Deterministic Guardrails）

- 用状态机约束流程，不让它乱走
- 工具白名单 + 参数校验
- 结构化输出 + schema 强校验
- 关键步骤插入 human-in-the-loop

一句话：用确定性的‘笼子’，关住概率性的‘猛兽’。模型负责开放式的判断，外面的工程负责把边界守死——这是今年生产落地的主旋律。



Harness 工程：模型之外那层‘驾驭’代码

光有模型 ≠ agent

- **Harness** = 包在模型外的‘运行时’：跑循环、执行工具、管上下文、设护栏
- 模型负责‘判断下一步’；harness 负责‘真正跑起来、守住边界’
- 同一个模型，harness 不同，能力与可靠性天差地别
- 框架（LangGraph 等）本质就是帮你写好这一层

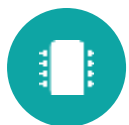
HARNESS（你写的工程代码）

- 主循环 while(任务未完成)
- 工具执行 + 结果回填
- 上下文组装 / 压缩
- 护栏：白名单 / 校验 / 人工确认
- 终止条件 + 重试 + 全程日志

↕ 调用

大模型 LLM — 只负责判断下一步

2026 的共识：agent 可靠性，七分靠 harness、三分靠模型。 换更大的模型，常不如把 harness 打磨好——循环、上下文、护栏、可观测，才是生产落地的命门。



拆开一个 Harness：一轮循环都做了什么

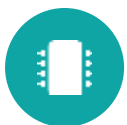
极简 agent harness 主循环（伪代码）

```
ctx = build_context(task, mem)
      # 上下文工程
for step in range(MAX_STEPS): # 终止
    out = llm(ctx)           # 判断
    if out.final: return answer
    if tool in WRITE_OPS:    # 护栏
        require_human_confirm()
    if tool not in ALLOWLIST:
        reject()             # 白名单
    res = run_tool(tool, args) # 执行
    log(step, out, res)       # 可观测
    ctx = compress(ctx + res) # 压缩
```

Harness 必做的六件事

- ✓ 循环与终止条件（防死循环）
- ✓ 工具执行与结果回填
- ✓ 上下文组装与压缩
- ✓ 护栏：白名单 / 校验 / 人工确认
- ✓ 错误重试与降级
- ✓ 全程日志，可回放定位

这段循环，就是‘感知→规划→行动→反思’的代码化身。框架（LangGraph 等）帮你写好这层；自研也行——但这六件事，一件都不能少。



前沿 · 运行时 · 工业参考实现

Harness 参考实现：Claude Code / Claude Agent SDK

同一套 harness 打包成库：Claude Agent SDK 就是驱动 Claude Code 的运行时，把上节‘六件事’做成可复用的参考实现。

Claude Agent SDK · 核心循环 (Python · 示意)

```
from claude_agent_sdk import query, ClaudeAgentOptions

opts = ClaudeAgentOptions(
    allowed_tools=["Read", "Grep", "Bash"], # 白名单护栏
    permission_mode="acceptEdits", # 写操作可控
    mcp_servers=cfg, # 接外部工具
    max_turns=40, # 终止条件
)

# 同一套循环: gather → act → verify → repeat
async for msg in query(prompt=goal, options=opts):
    handle(msg) # 全程可观测 · 可 resume/fork
# 上下文自动压缩 (compaction), 跨多窗口长跑
```

同一套 harness, 两种入口

Claude Code (CLI, 给人用) 与 Agent SDK (库, 给程序用) 共享同一运行时——‘接口之别, 非能力之别’。

‘六件事’, SDK 全包

循环与终止 · 工具执行回填 · 上下文压缩 · 白名单与人工确认 · 重试 · 日志与 resume/fork。

工业落地：驱动 Claude Code 编码 agent 与 CI/CD；长跑用‘初始化 + 增量’或 planner-generator-evaluator 模式；MCP 接外部系统。

参考实现的价值：你不必从零写 harness——直接复用‘gather→act→verify→repeat’这套被验证的运行时，把精力放在工具、上下文与护栏的取舍上。



Loop Engineering: 不再逐条敲提示词，而是设计‘循环’

从‘敲提示词’到‘写循环’

- **核心：**你不再当那个一遍遍敲提示的人，而是设计一套系统替你敲 (Addy Osmani)
- 一个 loop = 一个递归目标：定好目的，让 AI 反复迭代直到完成
- 系统自己：发现任务 → 派给 agent / 子 agent → 校验 → 存状态 → 决定下一步
- **按计划或直到达标：**可挂在 cron 上定时跑，也可一跑到底
- **很新：**源于 2026.6 的讨论，是趋势、但还不是成熟方法论

抽象阶梯 · 越往上越抽象 ↑

循环工程 Loop Engineering
设计驱动 agent 的系统

← 本节主题

Harness 工程 Harness Engineering
装备单次 agent 运行

上下文工程 Context Engineering
把对的信息喂进上下文

提示词 Prompting
一句一句敲指令

提示词 → 上下文 → Harness → 循环：层级一路上移。

Harness 管‘一次 agent 运行’，Loop 管‘不停驱动 agent、自我喂料、按目标迭代’——活没消失，只是上移了一层：你来决定‘这个循环该往哪跑’。



前沿 · 运行时

怎么落地一个循环：发现 → 执行 → 校验 → 决策

一个最小循环（伪代码 · 按计划自动跑）

```
goal = read("VISION.md") # 目标锚点
while not goal_met(goal):
    task = discover_next() # ①发现
    res = run_agent(task) # ②执行
    if not verify(res): # ③校验
        # 关键：放一个能说‘不’的关卡
        retry_or_handoff()
    persist(res) # ④持久化
    if hit_boundary(): pause() # 触边界
```

循环里最该放的东西：一个会‘说不’的关卡

测试 / 类型检查 / 真实报错——能把结果挡回去的硬信号。没有它，循环只是 agent 反复同意自己。

Ralph Wiggum Loop（极简范式）

让 agent 持续跑、反复尝试同一任务直到成功；成不成由‘外部验证’说了算，而不是它自评。

落地件套： VISION.md 锚目标 · AGENTS.md 定规则 · cron 定时 · Claude Code 里 /goal /loop 触发

先拿低风险任务起步（升级依赖、修测试），循环里务必放一个能‘说不’的校验，再逐步放权。也要盯住 token 开销——它会随‘跑多久、跑多少轮’迅速放大。



Loop 参考实现：AlphaEvolve / OpenEvolve

自驱动的外层循环：AlphaEvolve (DeepMind) 用‘生成→评测→选择’反复进化，自动评测器当‘会说不’的关卡，跑出真实工业收益。

AlphaEvolve 进化循环 (四件套·示意)

```
db = ProgramDatabase(seed=program) # 程序库·进化选择
while not budget_done:
    ctx = prompt_sampler(db) # ①采样 好+多样
    cand = llm_ensemble(ctx) # ②生成 LLM 提变体
    score = evaluators(cand) # ③评测 会说不
    db.add(cand, score) # ④入库 留更优解
best = db.top() # 跑数百万次 → 逼近/超越基线

# OpenEvolve: 开源参考实现
evolve.run(iterations=1000)
```

四件套 = 一个闭环

采样 Prompt Sampler → 生成 LLM Ensemble → 评测 Evaluator Pool → 入库 Program Database；控制器异步编排、吞吐拉满。

真实工业收益

回收约 0.7% 数据中心算力（已上线>1年）· 优化芯片电路 · Gemini 训练提速约 1%
· 破 56 年矩阵乘法纪录。

OpenEvolve · 开源参考实现：忠实复刻四模块、OpenAI 兼容 API、断点续跑；
evolve.run(iterations=1000)，拿来即可改。

为什么是‘循环工程’范例：它不靠更大的模型，而靠一个‘会说不’的外层循环——目标可量化、每步必验证、只留更优解。这正是 Loop Engineering 的工业级答卷。



自进化 · ① 是什么

自进化：让 agent 「边跑边变强」

今天的 agent 大多「出生即定型」：prompt 写死、工具固定、不会从错误里成长。自进化 = 把「改进」这件事自动化成一个闭环。



本质 = 把「评估 → 改进」做成自动化反馈回路 (2026 的 arXiv 综述把各种做法统一进这一框架)。

其实你早在用最朴素的版本：看人审日志 → 改 prompt → 发版。把这个手工循环自动化，就是自进化的起点。



自进化：四种落地范式 + 安全护栏

自进化不是「让模型随便改自己」——从最轻的反思入库，到最重的源码级自改，风险递增，务必配套护栏。

落地范式（由轻到重）

- **反思入库 Reflexion**：失败后总结教训写进记忆，下次先检索
- **自博弈 MAE**：一个模型分饰 Proposer/Solver/Judge，用 RL 自我对弈变强
- **轨迹 RFT**：用成功轨迹 + 奖励做强化微调
- **源码级自改（2026）**：MOSS / Self-Harness——找自己的弱点、重写模块、跑测试验证

安全护栏（生产铁律）

- **离线提议**：用 meta-agent 离线提改动、走评审流水线，别让 agent 实时改自己
- **先有地基**：干净的日志 + 明确的成功指标，才谈得上自动改进
- **强验证**：每次改动都过「会说不」的测试 / 回归 / 人审
- **低风险起步**：先在低风险任务放权，盯住成本（token 开销会涨）

一句话：自进化 = 把「评估 → 改进」做进系统本身。社区最担心「递归自我改进」失控——**务实做法**：离线提议、强验证、人把最后一关。



递归自我改进：让 agent 「越改越会改」

比自进化更进一步：RSI 改进的是「改进自己」这件事本身——每一轮都让下一轮更快，理论上从线性滑向指数。



为什么被高度关注（理论）

- **智能爆炸假说**：自我改进滑成失控正反馈（I.J. Good, 1965）
- **软起飞 vs 硬起飞**：每轮收益不衰减，能力或在极短时间剧增
- **监督失灵**：改进快过人类评估，安全反馈回路就断了
- **业界声音**：多家前沿实验室认为「智能爆炸」值得认真对待

现实在哪（2026）

- **尚无实证**：迄今没有持续、开放式的自我改进案例
- **普遍规律**：实验多在 1-2 轮后收益递减、趋于平台
- **三道闸**：算力、数据、可验证性都在限制「软件式爆炸」
- **真实但有界**：AlphaEvolve 把 LLM 训练提速约 1%，是「AI 改进 AI」但有限

一句话：自进化 = 把活干得更好；递归自我改进 = 让「改进自己」越来越快，这正是社区最担心失控的剧本——**务实立场**：离线提议、强验证、限制改动训练基建的权限、人把最后一关（与上页护栏一脉相承）。



前沿 · 安全

连上世界，也连上了攻击面



工具投毒 Tool Poisoning

恶意 MCP server 在工具描述里藏指令，操纵 agent 行为



提示注入 Prompt Injection

网页/文档/邮件里埋指令，agent 一读到就被劫持



过度授权

agent 手握能删库/转账的权限，一旦被骗后果严重

防御要点

- 可信网关：只连白名单 server
- 全程审计：每次工具调用可追溯
- 最小权限：能只读就不给写权限
- 副作用需确认（写/发/删）
- 输入隔离：把外部内容当‘数据’不当‘指令’

实操经验：把工具按‘读’与‘写/发/删’分级——读操作可放手，写操作默认要人确认。这条规则能挡住绝大多数事故。



智能体经济：当 agent 开始「自己花钱」

agent 不再只是查信息——它开始**发现商品、获授权、付款、对账**。「谁是顾客」正在改变：从说服人，变成满足 agent 的筛选条件。

结账层

Checkout

ACP (OpenAI+Stripe, 驱动 ChatGPT Instant Checkout) **UCP** (Google+Shopify/Walmart, 2026.1 NRF 发布)

授权信任层

Authorization

AP2 (Google, 密码学「mandate」= 用户签名授权的意图) **Visa TAP** (卡组织侧可信代理)

结算层

Settlement

x402 (Coinbase, 复用 HTTP 402 跑稳定币微支付, Linux 基金会托管) **MPP** (Stripe/Tempo, 会话预授权额度)

关键模式：agent 从不填卡号——而是出示「限额 / 限商家 / 一次性」的临时凭证 (SPT / 签名 mandate)。落地分流：消费购物走卡组织，机器对机器（算力/数据）走稳定币。

agent 当顾客，重塑零售：店铺的活从「说服人」变成「满足 agent 的筛选条件」（Adobe 测得 gen-AI 导购流量同比暴涨）。**但这也是合规与欺诈风险最集中处，目前仍早期——宜小步试点。**



Computer Use：像人一样操作软件

看截图、移鼠标、敲键盘——直接操作没有 API 的老系统（ERP 录入、税务门户、发票处理），正在蚕食传统 RPA 的地盘。

• Claude Computer Use

截图 + 鼠标 + 键盘，可移植、不绑 OS，跨 VM / 容器都能跑

• OpenAI Operator / CUA

以浏览器为主；Codex 后台 Computer Use 上桌面（2026.4）

• Google Gemini CU

源自 Project Mariner，浏览器锚定、感知 DOM

OSWorld：14% → ~72%

2024 → 2026，逼近人类基线（~72%）

单任务成本：\$0.5–1.5 → \$0.05–0.15

两年降一个数量级

务实原则：有 API/工具就别用像素（WebMCP 让 agent 直接调工具，短路掉慢循环）；代价是慢、易错、基准能被刷（2026.4 伯克利）。务必沙箱 + 细粒度权限 + 关键动作人工确认，高风险流程暂别全放。



智能体做科研：自己提假设、跑实验、写论文

不止查资料：agent 能自己提假设、设计并跑实验、分析数据、写论文与自评审——把科研全流程自动化。

● Sakana The AI Scientist

端到端提假设→跑实验→写论文→自评审；其论文 2025 年首次通过 ICLR workshop 同行评审

● Google AI co-scientist

Gemini 2.0 多智能体，「生成-辩论-进化」提假设；已在药物重定位、肝纤维化新靶点等验证

● DeepMind AlphaEvolve

Gemini + 自动评测器的进化式编码 agent；发现 4x4 矩阵 48 次乘法，56 年来首超 Strassen

首篇全 AI 论文通过同行评审

ICLR 2025 workshop，均分 6.33（约前 45%）

矩阵乘法：49 → 48 次乘法

AlphaEvolve 打破 Strassen 保持 56 年的纪录

现实定位：AI 已在优化训练 AI 的算法与基建（AlphaEvolve），这是「递归自我改进」的现实入口；但 2026 仍是「加速器，非替代者」——质量参差、评审可被刷、真正的新颖洞见仍偏串行。务实做法：放在能复跑、能证伪的环节，让 AI 扩大假设空间、人来定方向把关。



把所有技术拼起来：一个机票客服 Agent

场景 · 用户说：“下周三去上海的机票帮我退了，改订高铁，要靠窗。”

1 感知 Perceive

解析意图：退机票 + 改高铁 + 靠窗
query_order 查到 MU5102
6/17 北京→上海，已出票

2 规划 Plan

拆成：退票 → 搜高铁 →
订靠窗 → 发确认
命中‘退改签’Skill 读规则

3 行动 Act

按计划调 MCP 工具执行：
refund_ticket → search_train
→ book_train（写操作先确认）

4 反思 Reflect

核对每步结果；若高铁售罄
或退票失败 → 回到规划
换车次/时间，再试一遍

🔄 **闭环：**没达成就带着‘反思’回到‘感知’重来——这正是 ReAct 的想一步·做一步·看一步
Agent = LLM 负责判断下一步，**Harness** 负责把这四步真正跑成循环、加护栏、可重试。

接下来两页：先看这一单的 ReAct 真实轨迹（一步步怎么走），再回头数一数到底用上了哪些技术。



真实轨迹： ReAct 一步一步怎么走

行动（工具调用） → 观察（返回结果）		阶段	用到的技术
1	query_order(“下周三 上海”) → MU5102 6/17 已出票	感知	MCP 工具
2	命中‘退改签’Skill，读规则 → 起飞前可退、扣 5%	规划	Agent Skill
3	refund_ticket(MU5102) II → 用户确认后已退 ¥1330	行动	MCP · 人工确认
4	search_train(京→沪 6/17 上午) → 命中 G1 等 6 班高铁	行动	MCP 工具
5	读 Memory：偏好‘靠窗·直达’ → 锁定 G1 靠窗，无需再问	规划	Memory 记忆
6	book_train(G1, 靠窗) II → 确认后出票，发确认通知	行动	MCP · 人工确认

看每一步都‘先想再做、做完看结果’： Skill 给规则、MCP 干活、Memory 记偏好、写操作有人工确认兜底——四者合奏，才走完这一单。



实战 · 综合示例

这一个任务，用上了哪些技术

ReAct 运行范式

想一步·做一步·看一步，每步依据真实反馈再决定下一步

Agent Skill

把‘退改签’的规则与步骤打包，命中任务即加载

Harness 运行时

跑主循环、回填结果、重试与终止，把四步串成闭环

MCP 工具

query_order / refund_ticket / search_train / book_train 真正执行

Memory 记忆

记住该用户‘靠窗·直达’偏好，这次直接复用、不再追问

护栏 + 人工确认

退票/订票等写操作先暂停，等用户点头才执行

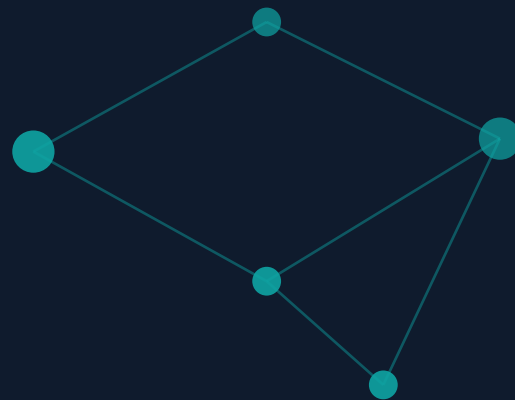
一句话收束：感知→规划→行动→反思 是骨架，ReAct 是迈步方式，MCP/Skill/Memory 是手脚与记性，Harness + 护栏 让它在生产里真正跑得起来。

06

REALITY & ADOPTION

现实与落地

热闹之后，讲点不那么性感的：落地鸿沟、踩坑经验、评估与路线图。





冷静的数据

为什么很多 agent 项目卡住了

62%

在试验 agent

<25%

真正规模化进生产

>40%

到 2027 底或被叫停

为什么会被叫停 (Gartner)：成本上升、业务价值不清晰、风险控制不足。启示——别为了 agent 而 agent，选‘算得清 ROI、控得住风险、好评估’的场景再投入。



实战经验

我们（会）踩的七个坑

1 任务没拆清 → agent 原地打转、烧 token

2 工具太多 / 描述含糊 → 选错工具

3 没有终止条件 → 死循环停不下来

4 上下文塞太满 → 忘中间、变贵变慢

5 没有评估 → 改动后不知是变好还是变坏

6 一上来就多 agent → 复杂度爆炸

7 把写操作交给 agent 自动执行 → 出事



评估与可观测性：没有它就无法维护

评估什么（维度）

- 端到端任务成功率（最终有没有办成）
- 轨迹质量（每一步是否合理）
- 工具调用准确率（选对没、参数对没）
- 成本 & 延迟（一个任务多少钱多少秒）
- 安全合规（有没有越权/泄露）

怎么评 + 怎么看

- 黄金任务集：每次改动跑回归
- LLM-as-judge：让模型给轨迹打分
- 人审高风险样本
- 可观测性：记录每步 thought/action/obs，可回放定位‘哪步错’

洞见：没有评估的agent = 不可维护。评估体系要先于规模化——先能度量，再谈放量。



该不该上？上哪种？一张决策树



起步建议：选一个高价值、容错高、易评估的场景做 PoC——先证明价值，再谈规模化。



带走就能用

上线前的风险控制清单



明确任务边界与终止条件（防死循环）



只连白名单 MCP server，全程审计可追溯



黄金集回归 + 上线后监控成功率/成本/延迟



数据脱敏，敏感信息不进第三方上下文



工具最小权限；写/发/删操作需人工确认



输出做结构化校验（schema 强约束）



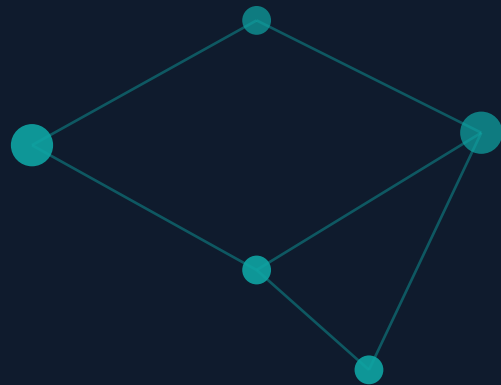
高风险步骤插入 human-in-the-loop



把外部内容当‘数据’而非‘指令’（防注入）

带走这五句

总结



- 1 智能体 = 会自主干活的大模型 — 规划 + 记忆 + 工具 + 反思 + 执行循环
- 2 2026 关键词不是‘更强模型’ — 而是工程化、协议化、可靠性与安全
- 3 协议正在成为标准 — MCP 连工具(纵向) + A2A 连 agent(横向)
- 4 落地铁律 — 能用确定性方案就别上 agent；先单后多；评估先于规模化；写操作要人确认
- 5 先选对场景 — 高价值、容错高、易评估，做 PoC 证明价值

资源：MCP / A2A 官方规范 · ReAct、Reflexion、Toolformer 论文 · 各 Agent 框架文档。 谢谢，欢迎提问。